

Georgia Institute of Technology

CS 4365/6365: Introduction to Enterprise Computing

Summer 2026

Project Checkpoint 1 Report

Name: Daniel Oh

Project Name: GraphAtlas

Context and Related Work:

Project Plan (Plan)

Modern enterprises often struggle with having data buried across multiple platforms such as Slack threads, Jira tickets, and Confluence wikis. While traditional keyword search engines and RAG approaches are effective at retrieving single documents, they cannot find structural relationships or dependencies across multiple documents and platforms. For instance, LangChain and Vector RAG pipelines often hallucinate or break when being tasked to do so. Graph solutions do exist, but data engineers must manually map information, a process that is time-consuming, tedious, and prone to human error.

The solution — an autonomous graph pipeline that allows for relationship-aware reasoning, an improvement over static document retrieval. The system will be able to intake multiple data streams in real time, use LLMs to extract entities and their relationships, and continuously merge them into one unified Neo4j knowledge graph. Additionally, a GraphRAG interface will be available for users to ask questions and have a conversation with the system.

Project Deliverables

Add the deliverables planned for the project at the end of the semester and remove this line.

Deliverable	Description	Technical Stack

Streaming ingestion engine	Data pipeline that takes in unstructured data from multiple sources and chunks it for processing with metadata	Python, Apache Kafka, Unstructured.io
Entity & relationship pipeline	Takes in the standardized text chunks and prompts the LLM to identify key entities and how they all relate with each other	LlamaIndex, Llama LLM API
Graph & vector storage	Layer that stores relationships (graph) and semantic meanings (vectors), allowing the system to semantic search and traverse the graph for context	Neo4j, Qdrant
GraphRAG interface	Interface where users can query the system conversationally	Streamlit, LlamaIndex Graph Query Engine
Evaluation	Document showing the system's accuracy statistics and overall performance	RAGAS, Prometheus

Project Milestones

Check point	Milestone	Technical Scope & Deliverables	Work Splitup	Status
Check point 1	Idea and timeline proposal	Defined the scope of the project as well as technologies involved. Deliverables include the report with the idea and initial timeline as well as	This project is being completed solo.	Completed

		the initialized GitHub repository.		
Check point 2	Data collection & infrastructure setup	Collect Slack and Jira data and initialize the local, containerized environment. Additionally, build the Python script necessary to input data into Kafka and implement parsing logic to standardize the text chunks. Deliverables include a .yaml file deploying Kafka and Neo4j along with the Python script described.	This project is being completed solo.	To do
Check point 3	LLM entity extraction & graph setup	Set up LlamaIndex with a free LLM API like Meta's Llama, write the logic around turning text chunks into actual entities with relationships, and populate the databases. Deliverables include Python extraction engine, a Neo4j instance, and Qdrant instance.	This project is being completed solo.	To do

Check point 4	GraphRAG query engine setup	Write the logic around searching Qdrant for semantic meaning, extracting relevant nodes from the graph, and sending this context to the LLM. Deliverables include a working CLI engine for users.	This project is being completed solo.	To do
Check point 5	UI setup and final delivery	Built the front-end user interface and run evaluations to prove the system's accuracy. Deliverables include a full Streamlit interface as well as the final report and presentation,	This project is being completed solo.	To do

Current Progress Report (Match)

During the last two weeks, I was able to find an idea that I felt comfortable with and find a technological stack that fits given the project's goal. Most of the time was spent brainstorming ideas, so picking technologies and mapping out the full plan for the project was rushed, but I believe things can evolve as I continue to work on this project.

For the next two weeks, I first plan to set up the local architecture, configure Apache Kafka, Neo4j, and Qdrant. I then plan to synthesize my data (Slack JSON, Jira CSV, etc.) and create a Python script that handles raw data using Unstructured.io. Lastly, I plan to establish the Kafka pipeline, including having the script publish the new structured text chunks into Kafka topics. I plan to have factual evidence for this part specifically, maybe a little script that prints the chunks that are put into each Kafka topic to prove the pipeline was set up properly.

Since this is the initial milestone chart and original plan, there is no base to compare to, but I'm sure future checkpoints will refer back to this plan.

Supporting Evidence (Factual)

<https://github.com/danioh04/GraphAtlas>

- <https://github.com/danioh04/GraphAtlas/tree/main/reports>

Skill Learning Report

Self-Evaluation

1. Plan: 110

I think in this checkpoint I've successfully addressed a problem space, a solution, and a reasonable timeline to complete this project, and in future checkpoints I'm looking forward to learning how to plan in more detailed ways.

2. Match: 110

I completed all checkpoint 1 objectives as planned, and I'm learning how to execute what I plan to do efficiently.

3. Factual: 100

I was able to provide the GitHub repository, and it is all I planned for in this checkpoint evidence-wise, but next checkpoint I will aim to have more factual evidence, including reasons why I chose the technology stack that I did.

LLM Feedback:

1. Project Plan (Plan): 110

"Your solid problem space definition and roadmap earned this score, but explicitly including a formal risk analysis and mitigation strategy will push it closer to the 120% maximum." (Gemini)

2. Progress Report (Match): 100

"You successfully met all core objectives by establishing the scope and tech stack, and you can maintain or exceed this score by ensuring your future execution strictly aligns with these planned choices." (Gemini)

3. Supporting Evidence (Factual): 90

“Providing the GitHub repository links earned you points, but the score is heavily limited because you lacked concrete visual artifacts of your progress.” (Gemini)

Actionable Suggestions:

“To improve your scores for the next checkpoint, incorporate a formal risk mitigation strategy into your plan and provide concrete visual evidence like screenshots of your initialized local environment.” (Gemini)